

# Керівництво з патернів проектування

---

## Вступ

Патерни проектування - це типові рішення часто зустрічаємих проблем у розробці програмного забезпечення. Вони допомагають створювати більш гнучкий, розширюваний та підтримуваний код.

## Класифікація патернів

Патерни зазвичай поділяються на три основні групи:

1. Породжувальні (Creational)
2. Структурні (Structural)
3. Поведінкові (Behavioral)

## 1. Породжувальні патерни

### 1.1 Singleton (Одинак)

Гарантує, що у класу є лише один екземпляр, і надає глобальну точку доступу до нього.

```
class DatabaseConnection {
    private static instance: DatabaseConnection;
    private connection: any;

    private constructor() {
        this.connection = this.initializeConnection();
    }

    public static getInstance(): DatabaseConnection {
        if (!DatabaseConnection.instance) {
            DatabaseConnection.instance = new DatabaseConnection();
        }
        return DatabaseConnection.instance;
    }

    private initializeConnection() {
        // Логіка підключення до бази даних
        return {};
    }

    public getConnection() {
        return this.connection;
    }
}

// Використання
const db1 = DatabaseConnection.getInstance();
```

```
const db2 = DatabaseConnection.getInstance();
console.log(db1 === db2); // true
```

## 1.2 Factory Method (Фабричний метод)

Визначає інтерфейс для створення об'єкта, але дозволяє підкласам вибирати, який клас інстанціювати.

```
interface Transport {
    deliver(): void;
}

class Truck implements Transport {
    deliver() {
        console.log("Доставка товарів вантажівкою");
    }
}

class Ship implements Transport {
    deliver() {
        console.log("Доставка товарів кораблем");
    }
}

abstract class LogisticsFactory {
    abstract createTransport(): Transport;

    planDelivery() {
        const transport = this.createTransport();
        transport.deliver();
    }
}

class RoadLogistics extends LogisticsFactory {
    createTransport(): Transport {
        return new Truck();
    }
}

class SeaLogistics extends LogisticsFactory {
    createTransport(): Transport {
        return new Ship();
    }
}
```

## 1.3 Builder (Будівельник)

Дозволяє створювати складні об'єкти покроково.

```

class Computer {
    private cpu: string = '';
    private ram: number = 0;
    private storage: string = '';

    setCPU(cpu: string): Computer {
        this.cpu = cpu;
        return this;
    }

    setRAM(ram: number): Computer {
        this.ram = ram;
        return this;
    }

    setStorage(storage: string): Computer {
        this.storage = storage;
        return this;
    }

    build(): string {
        return `Комп'ютер: CPU ${this.cpu}, RAM ${this.ram} ГБ, Сховище ${this.storage}`;
    }
}

// Використання
const gaming = new Computer()
    .setCPU('Intel i9')
    .setRAM(32)
    .setStorage('1TB SSD')
    .build();

```

## 2. Структурні патерни

### 2.1 Adapter (Адаптер)

Дозволяє об'єктам з несумісними інтерфейсами працювати разом.

```

// Старий інтерфейс
class EuropeanSocket {
    voltage220V(): string {
        return '220V';
    }
}

// Новий інтерфейс
interface USASocket {
    voltage110V(): string;
}

```

```

}

// Адаптер
class SocketAdapter implements USASocket {
    private europeanSocket: EuropeanSocket;

    constructor(socket: EuropeanSocket) {
        this.europeanSocket = socket;
    }

    voltage110V(): string {
        const voltage = this.europeanSocket.voltage220V();
        return '110V (конвертовано з ' + voltage + ')';
    }
}

```

## 2.2 Decorator (Декоратор)

Динамічно додає об'єкту нові обов'язки.

```

interface Coffee {
    cost(): number;
    description(): string;
}

class SimpleCoffee implements Coffee {
    cost(): number {
        return 5;
    }

    description(): string {
        return 'Проста кава';
    }
}

class MilkDecorator implements Coffee {
    private coffee: Coffee;

    constructor(coffee: Coffee) {
        this.coffee = coffee;
    }

    cost(): number {
        return this.coffee.cost() + 2;
    }

    description(): string {
        return `${this.coffee.description()}, молоко`;
    }
}

```

```

class SugarDecorator implements Coffee {
    private coffee: Coffee;

    constructor(coffee: Coffee) {
        this.coffee = coffee;
    }

    cost(): number {
        return this.coffee.cost() + 1;
    }

    description(): string {
        return `${this.coffee.description()}, цукор`;
    }
}

```

### 3. Поведінкові патерни

#### 3.1 Observer (Спостерігач)

Визначає залежність "один до багатьох" між об'єктами.

```

interface Observer {
    update(temperature: number): void;
}

class WeatherStation {
    private observers: Observer[] = [];
    private temperature: number = 0;

    addObserver(observer: Observer) {
        this.observers.push(observer);
    }

    removeObserver(observer: Observer) {
        this.observers = this.observers.filter(obs => obs !== observer);
    }

    setTemperature(temp: number) {
        this.temperature = temp;
        this.notifyObservers();
    }

    private notifyObservers() {
        this.observers.forEach(observer =>
            observer.update(this.temperature)
        );
    }
}

```

```
class TemperatureDisplay implements Observer {
    update(temperature: number) {
        console.log(`Температура: ${temperature}°C`);
    }
}
```

## 3.2 Strategy (Стратегія)

Визначає сімейство алгоритмів, інкапсулює кожен з них і забезпечує їх взаємозамінність.

```
interface SortStrategy {
    sort(data: number[]): number[];
}

class BubbleSort implements SortStrategy {
    sort(data: number[]): number[] {
        // Реалізація сортування бульбашкою
        return data.sort((a, b) => a - b);
    }
}

class QuickSort implements SortStrategy {
    sort(data: number[]): number[] {
        // Реалізація швидкого сортування
        return data.sort((a, b) => a - b);
    }
}

class Sorter {
    private strategy: SortStrategy;

    constructor(strategy: SortStrategy) {
        this.strategy = strategy;
    }

    setStrategy(strategy: SortStrategy) {
        this.strategy = strategy;
    }

    performSort(data: number[]): number[] {
        return this.strategy.sort(data);
    }
}
```

## ВИСНОВОК

Патерни проектування:

- Надають перевірені рішення типових задач
- Покращують архітектуру та читабельність коду
- Полегшують комунікацію між розробниками
- Допомагають створювати більш гнучкі та розширювані системи

**Важливо!** Застосовуйте патерни свідомо, тільки коли вони дійсно вирішують конкретну проблему у вашому проекті.

---

Керівництво допоможе вам ефективніше використовувати патерни проектування!